



第七届全国校园人工智能算法精英大赛-算法应用赛
智慧社区 省赛技术报告

基本信息：

队伍名称	Defvoid
参赛编号	AIC-2025-11673086
参赛队员	韩硕、杜浩、朱宇
摘要	本项目旨在解决城市社区治理中的人力管控难、安防效率低、车辆管理乱等痛点，基于ROS平台和Gazebo仿真环境，构建了一套全自动的智慧社区巡检机器人系统。系统涵盖了仿真环境构建、机器人建模、基于GMapping的SLAM建图、基于AMCL和move_base的自主精准导航、定点图像采集，以及离线YOLOv5行人检测与车牌识别。通过“采集与识别解耦”的架构，确保了导航的稳定性和识别的准确性，最终实现了机器人在仿真社区中按预设路线巡逻、精准停靠、并自动完成人员类别判断和车牌号码识别的全部要求。

目录

1. 引言	1
1.1 赛题理解与背景分析	1
1.2 核心挑战与解决思路	1
2. 系统总体设计	3
2.1 设计目标	3
2.2 系统架构图与技术路线	3
3. 核心技术与模块实现	5
3.1 仿真平台搭建	5
3.2 软件核心框架与开发环境	5
3.3 关键模块一：仿真与基础工具层	6
3.4 关键模块二：导航与数据采集层	8
3.4.1 功能描述：	8
3.4.2 算法原理与实现方法：	8
3.5 关键模块三：数据应用与输出层	16
3.6 本项目创新点	20
4. 测试与成果分析	20
4.1 功能测试	20
4.2 性能测试	21
4.3 成果展示	23
5. 总结与展望	25
5.1 工作总结	25
5.2 未来展望	25
附录：	26
核心代码片段：	26

1. 引言

1.1 赛题理解与背景分析

本赛题“智慧社区”着眼于当前城市社区治理中普遍存在的人力管控难、安防效率低、车辆管理乱等现实痛点，旨在探索以人工智能与机器人技术为核心的自动化解决方案。通过技术手段实现社区要素的智能感知与自主管理，是推动城市治理向“数字化、自动化、智慧化”演进的关键路径。

在省赛阶段，竞赛核心在于考核参赛选手在仿真环境下的技术实现能力。具体任务可分解为三个核心技术层次：

(1) 仿真环境搭建与机器人建模：作为系统功能实现的基础，本阶段的核心任务是基于给定地图，在ROS与Gazebo平台中构建一个具备真实物理属性和视觉特征的三维仿真场景，并完成机器人实体的建模与集成。该场景需准确还原社区的结构布局，并包含任务所需的人员与车辆标识物；机器人模型则需具备基本的移动能力与视觉感知模块，为后续的自主导航与图像采集提供载体。

(2) 建图及自主导航：本阶段旨在使机器人在仿真环境中实现从环境探索到自主运行的完整功能闭环。首先通过SLAM技术构建环境的二维地图，为导航提供空间依据；随后基于该地图，结合定位与路径规划算法，控制机器人沿预设路线稳定运行，并在指定任务点实现精准停靠，为图像采集创造条件。

(3) 视觉感知与智能识别：作为体现系统智能水平的关键环节，本阶段负责对机器人采集的图像信息进行有效处理与理解。通过在人员区域与车辆识别区采集图像，并运用目标检测与字符识别技术，实现对社区中行人类别与车辆牌号等关键信息的自动识别与统计，最终形成清晰的识别结果输出，完成对社区场景的智能化感知。

1.2 核心挑战与解决思路

在实现上述核心技术层次的过程中，我们主要面临以下三大核心挑战，并相应地制定了系统性的解决思路：

挑战一：如何构建高保真，可交互的仿真实验环境？

分析：仿真环境的真实性与准确性是所有算法测试的基础。挑战在于如何将二维地图精确转化为包含物理属性和视觉纹理的三维世界，并正确集成机器人模型与任务道具。

解决思路：我们采用Gazebo仿真平台与ROS集成的方案。首先，利用SDF格式定义仿真世界的物理参数，通过平面纹理映射还原赛场地图，并构建物理墙体作为边界。其次，使用URDF/Xacro模型定义机器人，通过添加差速驱动、摄像头与激光雷达的Gazebo插件，赋予其运动与控制能力。

挑战二：如何实现机器人在复杂环境中的精准定点停靠？

分析：任务要求机器人在特定坐标点精确停下以采集图像，这对机器人的定位与导航精度提出了极高要求。任何大的定位漂移都可能导致采集角度错误或任务失败。

解决思路：我们设计了一套基于SLAM建图与航点导航的解决方案。首先，使用GMapping算法结合激光雷达数据，构建高精度的二维环境栅格地图。随后，在已知地图上，通过AMCL算法进行实时精准定位，并利用move_base导航栈实现全局与局部路径规划，并且精准配置一套用于导航的参数来保证机器人的行走效果。我们编写了上层控制节点，按顺序向move_base

发送预设的航点目标，并在需要拍照的目标点暂停下来，确保机器人稳定停靠后执行图像采集，从而满足“定点采集”的要求。

挑战三：如何高效且准确的完成多目标，多批次的视觉识别任务？

分析：系统需在巡检查结束后，对采集自不同地点、包含不同目标的图片进行批量处理。这要求视觉模块既能处理多样化的识别任务，又能保证较高的准确率与处理效率。

解决思路：我们采用“采集与识别解耦”的架构和深度学习模型相结合的方案。机器人导航时，视觉系统仅负责在停靠点保存图片，不进行实时计算，以此保障导航稳定性。待所有图片采集完毕后，系统启动集中处理流程：首先使用我们训练好的模型进行行人社区内人外人员判断，然后再使用我们设计的一套高精度两阶段深度学习识别流程进行车牌的定位与字符识别。这种批处理方式简化了系统状态管理，便于集中资源进行高效、准确的离线识别，并统一输出所有结果。

2. 系统总体设计

2.1 设计目标

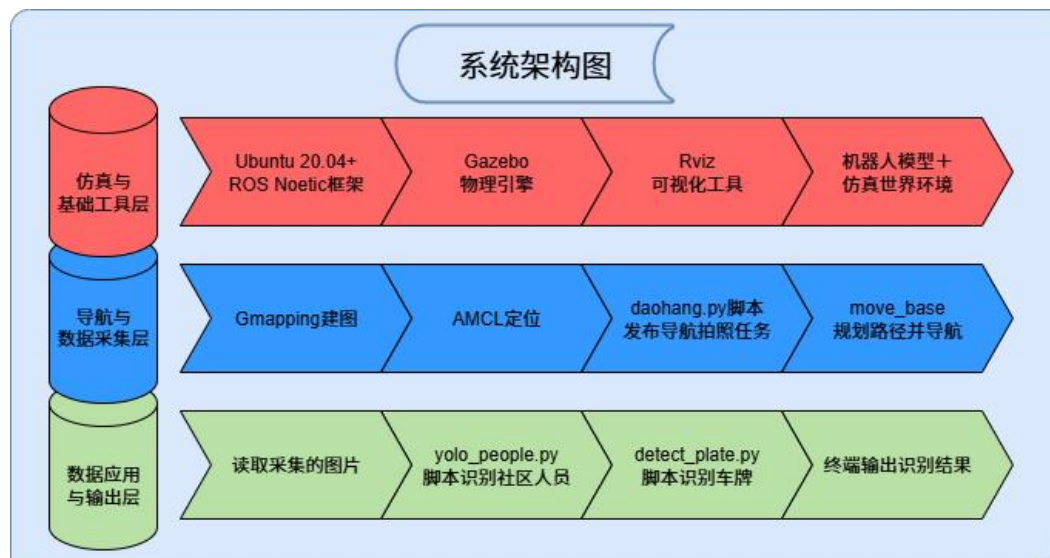
本系统的总体设计旨在构建一个高效、稳定且全自动的智慧社区仿真巡检机器人系统。围绕省赛核心要求，我们设定了以下具体设计目标：

- (1)**场景高保真还原**：在Gazebo中精确构建与赛题地图一致的仿真环境，包括场地布局、物理边界及任务标识物，为算法验证提供可靠环境基础。
- (2)**全流程模块化**：实现机器人从建模、自主建图、路径规划、精准巡航、定点停靠、采集图片、集中识别处理到输出结果的全过程模块化设计，便于逐步实现任务。
- (3)**任务可靠执行**：确保机器人能够稳定地在三个指定任务点（人员区域1、人员区域2、车辆识别区）停靠，并成功采集图像。
- (4)**识别准确高效**：在完成路径巡航后，系统能集中对采集的图像进行批量处理，准确识别出图像中的行人数量与车牌号码，并以清晰、规范的形式输出结果。

2.2 系统架构图与技术路线

2.2.1 系统架构图

系统采用模块化设计理念，将系统架构分为三层，以仿真与基础工具层构建运行环境和可视化支撑，通过导航与数据采集层实现环境感知、自主导航及数据采集，最终由数据应用层完成采集数据的智能分析与结果输出，形成从环境搭建到数据应用的完整智慧社区巡检流程。系统架构图如下所示：



2.2.2 技术路线

本项目遵循一种循序渐进的、分阶段实施的技术路线，其核心思想是“先搭建后编程，先感知后执行，分步验证，最终集成”。该路线确保了每个阶段的目标明确、成果可以验证，为后续阶段奠定了坚实可靠的基础。具体实施分为以下四个核心阶段：

阶段一：基础环境搭建

任务：在Ubuntu 20.04系统中配置ROS Noetic环境，创建car_simulation功能包，使用SDF和URDF/Xacro语言完成社区仿真世界与机器人模型的构建，并且创建launch文件car.launch用于在gazebo启动构建好的地图和机器人。

目标：在Gazebo中能够正常启动一个包含机器人及任务标识物的完整社区场景。

阶段二：建图

任务：在car_slam包中创建Launch文件slam.launch，启动Gazebo环境和gmapping SLAM节点，通过遥控操作机器人遍历整个环境，构建环境的二维栅格地图，并调用map_save工具对扫描出来的地图进行保存。

目标：生成并保存高精度的社区地图.pgm文件，为后续导航提供空间信息基础。

阶段三：自主巡航与图像采集

任务：(1) 使用map_server加载已保存的地图。
(2) 配置并启动move_base导航栈与amcl定位节点，并用一个launch文件test.launch来集成以上节点，用于一次性启动多节点。
(3) 编写daohang.py脚本，用于按顺序发送航点，并且导航至目标点后触发采集节点保存图像。

目标：实现机器人从起点出发，依次在人员区域1、人员区域2、车辆识别区自动停靠并采集图像，结束后回到待命状态。

阶段四：智能识别与结果汇总

任务：在机器人完成所有航点巡航后，该节点读取所有保存的图像，使用yolo_people.py脚本调用训练好的模型检测社区内外人员，然后再启动detect_plate.py脚本调用两阶段识别流程对车牌进行定位和识别。

目标：清晰输出各区域的检测结果，判断社区内外人员以及识别车牌号，完成整个智慧社区巡检任务。

3. 核心技术与模块实现

3.1 仿真平台搭建

本项目的仿真平台以“Ubuntu 20.04+ROS Noetic+Gazebo+Rviz”为核心工具链，构建机器人巡检所需的虚拟运行环境，实现仿真场景、机器人运动与数据交互的协同，为后续建图、导航及视觉识别任务提供基础支撑。

3.2 软件核心框架与开发环境

3.2.1 核心编程语言

Python作为唯一开发语言，覆盖系统全模块开发：编写导航任务调度脚本（如daohang.py，实现多航点自动巡航与定点拍照）、视觉识别处理脚本（yolo_people.py人员检测、detect_plate.py车牌识别）、ROS节点逻辑（如图像订阅与保存节点）。依托Python丰富的库生态，快速实现数据处理、图像操作、深度学习推理等功能，兼顾开发效率功能完整性。

3.2.2 核心架构

（1）机器人开发框架：ROS Noetic

提供导航与建图核心组件，包括gmapping（SLAM 建图，生成二维栅格地图）、amcl（自适应蒙特卡洛定位，实现机器人在已知地图中的精准定位）、move_base（导航栈，集成全局规划器与局部规划器，实现路径规划与避障）、map_server（加载与发布已保存的地图）。通过catkin_make工具构建功能包（如car_simulation仿真包、car_slam建图导航包），管理模块依赖与编译流程。

（2）深度学习框架：PyTorch

加载与推理 YOLOv5 模型（人员检测模型best.pt、车牌检测模型plate_detect.pt）及车牌识别模型（plate_rec_color.pth），支持 GPU 加速（自动适配 CUDA 设备），提升模型推理速度，满足批量图像处理需求。

（3）计算机视觉库：OpenCV 4.5

用于图像预处理（如letterbox缩放、BGR 与 RGB 格式转换、透视变换 four_point_transform）、视觉结果绘制（检测框、车牌角点、识别文字标注）、图像保存（cv2.imwrite），是连接原始图像与识别算法的关键工具；其cv2.cvtColor函数解决摄像头 BGR 格式与模型 RGB 输入需求的适配问题，cv2.warpPerspective函数实现车牌区域的正射校正，提升识别准确率。

（4）数据处理与存储库：Pandas

用于整理视觉识别结果（如人员分类计数、车牌信息：图像名、车牌号、置信度、坐标），生成plates.csv文件实现数据持久化，便于结果查看与分析。NumPy 用于数值计算（如车牌角点坐标处理、惯性矩阵参数计算、识别置信度统计），提升数据运算效率。

（5）其他辅助库

argparse用于解析命令行参数（如模型路径、图像输入 / 输出目录、是否识别车牌颜色），增强脚本灵活性与可配置性；pathlib与os用于文件路径处理（如遍历图像文件夹、创建输出目录）。threading用于多线程图像数据读写（如摄像头图像订阅与保存的线程锁控制），避免数据竞争。

3.3 关键模块一：仿真与基础工具层

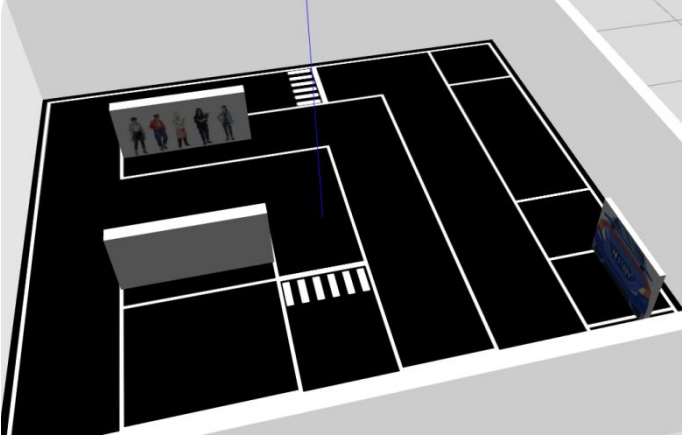
3.3.1 功能描述：

本模块是整个智能巡检系统的物理基础，包含仿真环境搭建和机器人建模两部分。具体而言，仿真环境搭建就是基于给定地图，在gazebo平台中构建一个包含真实物理属性和视觉特征的三维仿真场景，并准确还原社区的结构布局。而机器人建模则是负责完成机器人实体的建模与集成，赋予其移动能力和视觉感知模块，并集成差速驱动、摄像头与激光雷达的Gazebo插件，作为后续建图、导航和图像采集任务的载体。

3.3.2 算法原理与实现方法：

（1）仿真环境搭建

利用Gazebo仿真平台，通过SDF格式构建一个高保真、具备物理属性的三维世界。根据赛题地图尺寸，在SDF文件中精确构建尺寸为4.2*4.2*0.1的地面贴图模型（如下图）。在Gazebo软件中的Edit中的Building Editor功能中添加尺寸合适的墙壁，在Insert功能中手动添加官方提供的ro1、ro2、cp模型，设置好模型的位置，保存即可。



（2）机器人模型构建

采用Xacro宏语言将机器人模型分解为底盘、传感器、惯性矩阵等独立文件，并通过car.xacro文件进行聚合，模型组件的物理属性和连接关系如下表所示。

零件名称	形状	尺寸	作用
base_footprint	球体	半径 0.001m	机器人的地面参考点，用于提供全局坐标系基准
base_link	圆柱体	半径 0.1m，长度 0.08m	机器人最主要的物理实体，所有其他组件的安装基座
left_wheel	圆柱体	半径 0.0325m，长度 0.015m	提供驱动力，通过差速驱动插件控制机器人在仿真环境中移动
right_wheel	圆柱体	半径 0.0325m，长度 0.015m	提供驱动力，通过差速驱动插件控制机器人在仿真环境中移动
front_wheel	球体	半径 0.0075m	支撑底盘，保持车身平衡和离地间距
back_wheel	球体	半径 0.0075m	支撑底盘，保持车身平衡和离地间距

support	圆柱体	半径 0.01m, 长度 0.15m	抬高激光雷达, 提供其安装高度
laser	圆柱体	半径 0.03m, 长度 0.05m	采集环境点云数据, 用于 SLAM 建图和避障导航
camera_support_link	圆柱体	半径 0.01m, 长度 0.1m	抬高摄像头, 提供其安装高度
camera	长方体	0.01m*0.025m*0.025	采集图像数据, 用于人员和车辆的视觉识别任务

在它的物理属性定义中, 利用my_head.xacro中封装的宏, 精确计算并添加了球体、圆柱体和长方体等组件的惯性矩阵, 确保Gazebo仿真器能够进行准确的物理动力学计算。

```

1  <robot name="base" xmlns:xacro="http://wiki.ros.org/xacro">
2    <!-- Macro for inertia matrix -->
3    <xacro:macro name="sphere_inertial_matrix" params="m r">
4      <inertial>
5        <mass value="${m}" />
6        <inertia ixx="${2*m*r*r/5}" ixy="0" ixz="0"
7          iyy="${2*m*r*r/5}" iyz="0"
8          izz="${2*m*r*r/5}" />
9      </inertial>
10   </xacro:macro>
11
12   <xacro:macro name="cylinder_inertial_matrix" params="m r h">
13     <inertial>
14       <mass value="${m}" />
15       <inertia ixx="${m*(3*r*r+h*h)/12}" ixy="0" ixz="0"
16         iyy="${m*(3*r*r+h*h)/12}" iyz="0"
17         izz="${m*r*r/2}" />
18     </inertial>
19   </xacro:macro>
20
21   <xacro:macro name="Box_inertial_matrix" params="m l w h">
22     <inertial>
23       <mass value="${m}" />
24       <inertia ixx="${m*(h*h + l*l)/12}" ixy="0" ixz="0"
25         iyy="${m*(w*w + l*l)/12}" iyz="0"
26         izz="${m*(w*w + h*h)/12}" />
27     </inertial>
28   </xacro:macro>
29 </robot>

```

(3) 运动与感知插件集成:

运动能力: 在car.xacro中集成了Gazebo的差速驱动插件(gazebo/move.xacro), 将/cmd_vel话题的速度指令转换为左右驱动轮的角速度, 实现机器人的自主运动控制。

```

1  <robot name="my_car_camera" xmlns:xacro="http://wiki.ros.org/xacro">
2    <xacro:include filename="my_head.xacro" />
3    <xacro:include filename="my_base.xacro" />
4    <xacro:include filename="my_camera.xacro" />
5    <xacro:include filename="my_laser.xacro" />
6    <xacro:include filename="gazebo/move.xacro" />
7    <xacro:include filename="gazebo/laser.xacro" />
8    <xacro:include filename="gazebo/camera.xacro" />
9  </robot>

```

感知能力: 集成了Gazebo的激光雷达插件(gazebo/laser.xacro)和摄像头插件(gazebo/camera.xacro), 分别模拟传感器数据并发布到 ROS 话题, 作为后续建图和视觉识别模块的输入。

```

<!-- 核心插件 -->
<plugin name="gazebo_camera" filename="libgazebo_ros_camera.so">
  <alwaysOn>true</alwaysOn>
  <updateRate>0.0</updateRate>
  <cameraName>/camera</cameraName>
  <imageTopicName>image_raw</imageTopicName>
  <cameraInfoTopicName>camera_info</cameraInfoTopicName>
</plugin>

<!-- 激光雷达 -->
<plugin name="gazebo_rplidar" filename="libgazebo_ros_laser.so">
  <topicName>/scan</topicName>
  <frameName>laser</frameName>
</plugin>

```

3.3.3 难点与对策:

经测试发现，Gazebo世界中的部分障碍物模型缺失或未正确配置碰撞体积参数，导致机器人激光雷达无法感知（即无法被扫描到），且机器人可以直接穿越障碍物，造成仿真环境的物理特性严重失真，无法为导航算法提供真实可靠的测试基础。针对这一点，我们加入碰撞体积参数如下所示，示例为ro1区域立牌增加合适体积的碰撞体积参数，ro2和车牌立牌区域也如此：

```

<model name='ro1_0'>
  <static>1</static>
  <link name='link'>
    <visual name='visual'>
      <geometry>
        <mesh>
          <uri>model://ro1/meshes/ro1.dae</uri>
        </mesh>
      </geometry>
    </visual>
    <collision name='collision'>
      <geometry>
        <box>
          <size>0.8 0.1 1</size>
        </box>
      </geometry>
      <max_contacts>10</max_contacts>
      <surface>
        <contact>
          <code/>
        </contact>
        <bounce/>
        <friction>

```

3.4 关键模块二：导航与数据采集层

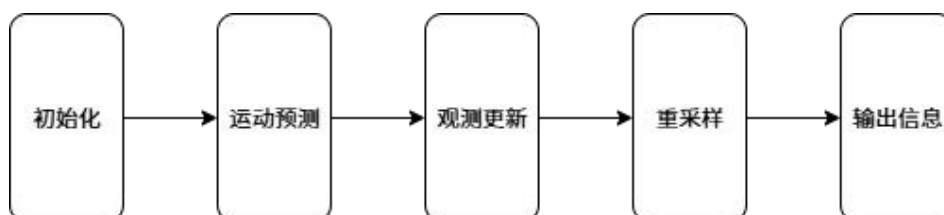
3.4.1 功能描述:

本模块基于已知地图实现机器人的精确定位与路径规划，包含AMCL定位、move_base导航系统和任务调度三个核心部分，是机器人自主移动和任务执行的基础。同时在导航过程中对图片进行采集，用于视觉识别。

3.4.2 算法原理与实现方法:

(1) AMCL自适应蒙特卡洛定位:

核心逻辑： AMCL自适应蒙特卡洛实现定位的原理主要分为五步如下所示：



第一步：初始化就是当最开始的时候，AMCL并不知道机器人在哪，所以它会在地图上随机撒一堆粒子，每个粒子包含(x坐标, y坐标, 朝向 θ , 权重w)信息，若设置了初始位置(initial_pose)那么粒子就集中在那一小片区域，否则会在全地图撒开。本代码设置了初始位置如下所示，生成的粒子会集中在指定坐标位置(1.6856, -1.6708, 1.5708)。

```
<param name="initial_pose_x" value="1.6856"/>
<param name="initial_pose_y" value="-1.6708"/>
<param name="initial_pose_a" value="1.5708"/>
```

第二步：运动预测就是根据里程计odom的数据可以知道机器人如何运动，移动了多少，然后再驱使所有粒子以相同的运动量进行移动，同时会加入随机噪声来考虑车轮打滑摩擦等等误差，用于模拟不确定性：

$$\begin{aligned}x_t &= f(x_{t-1}, u_t) + noise \\y_t &= f(y_{t-1}, u_t) + noise \\\theta_t &= f(\theta_{t-1}, u_t) + noise\end{aligned}$$

如下所示，设置了五个里程计噪声参数(odom_alpha1~5)，分别表示机器人转弯时角度误差，直行时方向和位移误差，转弯时位置误差以及横向误差。但最后一个误差参数常常用于全向轮，对差速机器人基本无影响。

```
<!--wuchacanshu-->
<param name="odom_alpha1" value="0.2"/>
<param name="odom_alpha2" value="0.2"/>
<param name="odom_alpha3" value="0.8"/>
<param name="odom_alpha4" value="0.2"/>
<param name="odom_alpha5" value="0.1"/>
```

第三步：观测更新就是观测每一个运动后的粒子的视角，每个粒子都将自己看到的视角与真实的激光雷达数据相比较，相似度高则权重大，相似度低，则权重小。

$$\omega_i = p(a_m | z_t)$$

a_m 表示当前激光测量数据， z_t 表示粒子的位姿， $p(a_m | z_t)$ 使用传感器模型计算概率。

第四步：重采样就是把所有粒子对应的权重标准化，高权重的粒子被保留，低权重粒子则被丢弃，按照此过程，这群粒子逐渐向机器人的真实位置靠拢。在这个过程中还使用了KLD-Sampling自适应粒子数量机制用于粒子数量控制，粒子云中粒子的数量越多，定位越精确但是运算速度变慢，若粒子数量少，则运算定位速度快，但是可能导致定位不精确丢失定位，所以要根据定位的不确定性自动调节粒子数，此机制可以很好得根据具体情况自动调节粒子数量，定义参数如下所示：

```
<!--zishiyinglizishu-->
<param name="min_particles" value="500"/>
<param name="max_particles" value="5000"/>
<param name="kld_err" value="0.05"/>
<param name="kld_z" value="0.99"/>
```

分别确定了粒子最少和最多数量，误差容忍和置信度参数。

第五步：输出最终位姿，AMCL常用加权平均来计算最终机器人位姿，并发布位姿(包括协方差)信息(/amcl_pose topic)和TF变换(map-->base_link)信息供导航系统使用：

$$\begin{aligned}\bar{x} &= \sum_{i=1}^N \omega_i x_i & \bar{y} &= \sum_{i=1}^N \omega_i y_i \\ \bar{\theta} &= \text{atan2}\left(\sum_i \omega_i \sin \theta_i, \sum_i \omega_i \cos \theta_i\right)\end{aligned}$$

(2) Move_base导航框架：

核心逻辑：集成全局规划器与局部规划器，实现完整的导航功能。全局规划器基于全局代价地图生成最优路径，局部规划器结合局部代价地图+实时传感器数据实现动态避障。需要对代价地图的参数进行合理配置以达到最好的避障效果。

代码文件：主要有两个参数文件分别用于配置局部代价地图和全局代价地图参数

以下为local_costmap_params.yaml文件内容，定义了机器人局部导航代价地图的空间范围、更新频率、参考坐标系与更新方式，让机器人能以自身为中心，实时构建一张半径约1.5米的动态障碍图，用于局部避障与轨迹规划。

以下为参数文件图片：

```

1 local_costmap:
2   global_frame: odom
3   robot_base_frame: base_footprint
4
5   update_frequency: 10.0
6   publish_frequency: 5.0      # 10 → 5 足够
7   transform_tolerance: 0.2
8
9   static_map: false
10  rolling_window: true
11  width: 6.0                  # 3 → 6 m, 视野翻倍, 规划不保守
12  height: 6.0
13  resolution: 0.05
14
15  plugins:
16    - {name: obstacle_layer, type: "costmap_2d::ObstacleLayer"}
17    - {name: inflation_layer, type: "costmap_2d::InflationLayer"}
18
19  obstacle_layer:
20    observation_sources: scan
21    scan:
22      sensor_frame: laser
23      data_type: LaserScan
24      topic: scan
25      marking: true
26      clearing: true
27      obstacle_range: 2.0
28      raytrace_range: 3.0
29
30  inflation_layer:
31    inflation_radius: 0.14     # 局部更小的半径, 避免整片半高代价
32    cost_scaling_factor: 5.0   # 衰减更陡: 离障碍20-30cm即快速降代价-加速

```

下表解释该参数文件中的关键变量的含义和用途：

参数名	含义	用途
update_frequency: 10.0	局部代价地图的更新频率（Hz），即每秒更新 10 次	远高于全局地图，因为局部地图需要实时处理传感器数据，快速响应突发障碍物，保证避障及时性
publish_frequency: 5.0	局部地图的可视化发布频率（Hz）	低发布频率以减少通信带宽消耗，5Hz 已足够满足可视化监控需求
transform_tolerance : 0.2	坐标系转换的容忍延迟（秒）	0.2 秒（比全局地图更严格），因为局部规划对实时性要求更高，需确保机器人位置与传感器数据的坐标匹配误差更小
obstacle_range : 3.0	激光雷达检测障碍物的有效范围	3米内的障碍物会被标记
raytrace_range 3.0	激光雷达射线追踪的范围	3米内无障碍物的区域会被标记为可通行

inflation_radius: 0.14	障碍物局部膨胀半径（米）	设置较小的膨胀半径，避免因过度膨胀导致局部区域被覆盖
cost_scaling_factor : 5.0	膨胀区域内代价的衰减系数，值越大，代价衰减越陡	配合小膨胀半径，让障碍物附近的代价快速降低，避免机器人因“过度谨慎”而频繁减速或转向，保证直线行驶的流畅性

该local_costmap配置的目标是，在保证实时避障安全的前提下，支持机器人高效跟踪直线：

①激光雷达实时标记/清除障碍物，适应动态环境；

②小半径膨胀 + 陡峭代价衰减，平衡了避障安全性和直线行驶的流畅性。

以下为global_costmap_params.yaml文件内容，定义了一个基于静态地图（map frame）的全局代价地图（global_costmap），用于规划器生成从当前位置到目标点的最优全局路线。

以下为参数文件图片：

```

1  global_costmap:
2    global_frame: map
3    robot_base_frame: base_footprint
4
5    update_frequency: 5.0      # 1.0 → 5.0, 路径更新更及时
6    publish_frequency: 2.0
7    transform_tolerance: 0.3   # 0.5 → 0.3, 减少时延保守
8
9    static_map: true
10   rolling_window: false
11   resolution: 0.05
12
13   plugins:
14     - {name: static_layer,    type: "costmap_2d::StaticLayer"}
15     - {name: inflation_layer, type: "costmap_2d::InflationLayer"}
16
17   inflation_layer:
18     inflation_radius: 0.13    # ↑ 推开全局路径，离墙更远
19     cost_scaling_factor: 5.0  # 衰减较缓，保持“推开”效果

```

下表解释该参数文件中的关键变量的含义和用途：

参数名	含义	用途
update_frequency: 5.0	全局代价地图的更新频率（单位：Hz）	提高更新频率可让地图更及时地反映环境变化，确保路径规划基于最新的环境状态
publish_frequency: 2.0	全局代价地图可视化发布频率（单位：Hz），即每秒向 ROS 话题发布2次地图数据	发布频率低于更新频率，是为了减少通信带宽消耗
transform_tolerance: 0.3	坐标系转换的容忍时间（单位：秒），即允许坐标系变换存在0.3秒的延迟	缩短容忍时间可减少因坐标变换延迟导致的误差，让机器人位置与全局地图的匹配更及时
inflation_radius: 0.13	障碍物的膨胀半径（单位：米）	参数调大，可让全局路径与墙壁、障碍物保持更远的安全距离，降低碰撞风险
cost_scaling_factor: 5.0	膨胀区域内代价的衰减系数	设为 5.0 属于中等衰减速度，配合膨胀半径，可在膨胀范围内保持

		较高的代价，同时避免过度膨胀导致路径可选范围过小。
--	--	---------------------------

该global_costmap配置的核心目标是：在已知静态环境中，构建高精度、低延迟的全局代价地图，同时通过障碍物膨胀确保路径与固定障碍保持安全距离：

- ①高频更新（update_frequency:5.0）和低时延容忍（transform_tolerance: 0.3）保证了地图的实时性；
- ②静态地图+膨胀层的组合，利用预先构建的环境信息，通过构建安全区提升了避障安全性；

（3）TrajectoryPlannerROS局部规划算法：
核心逻辑：在机器人的控制空间中进行采样，模拟生成多条短期可行的运动轨迹，通过多目标代价函数评选出最优轨迹，输出速度控制指令。通俗得说，在当前时刻，规划器不知道哪个速度最好，既能避障又能接近目标，所以规划器会尝试很多种不同的线速度+角速度组合，对每种情况都进行短时间的运动预测，模拟出这些情况的轨迹并计算每条轨迹的代价，然后选择代价最小的那条轨迹来执行。
配置参数：合适的参数能够优化机器人行走过程中的姿态，使机器人行走更加平稳，停靠的点位更加准确，也可以避免出现大幅度震荡导致压线以及到达目标点之后原地打转的现象，对于固定点位停靠进行图像采集任务十分重要，若机器人实际停靠的点位不准确，则可能导致机器人采集的图片拍不全最终进一步导致图片识别有误。

以下为base_local_planner_params.yaml文件内容，用于配置局部规划器的参数。

```

1 TrajectoryPlannerROS:
2   # ----- 1. 核心路径跟踪参数（强制贴合直线） -----
3   path_distance_bias: 32.0      # 优先贴合全局路径（直线），值越高直线性越强
4   goal_distance_bias: 0.8      # 降低目标点优先级，避免为赶目标偏离直线
5   heading_lookahead: 0.3       # 机器人朝向预判距离，聚焦路径方向
6   heading_scoring: true        # 开启朝向评分，确保机器人沿路径方向行驶
7   heading_scoring_timestep: 0.8 # 朝向评分时间步长，控制朝向调整频率
8   xy_goal_tolerance: 0.15      # 到达目标的XY允许误差（米），越小越精准
9   yaw_goal_tolerance: 0.16     # 到达目标的朝向允许误差（弧度），越小方向越准
10  latch_xy_goal_tolerance: false # 不锁定目标误差，确保全程严格跟踪
11
12  # ----- 2. 速度与转向限制（减少转向干扰） -----
13  max_vel_x: 0.7                # 最大前进速度（米/秒），根据机器人性能调整
14  min_vel_x: 0.1                # 最小前进速度，避免走走停停导致偏移
15  max_vel_theta: 1.2            # 最大旋转角速度（弧度/秒），限制频繁转向
16  min_vel_theta: -1.2           # 最小旋转角速度（负为反向旋转）
17  max_rotational_vel: 1.2        # 最大旋转速度（与max_vel_theta一致）
18  min_in_place_rotational_vel: 1.0 # 最小原地旋转速度，避免卡死
19
20  acc_lim_x: 1.0                # 线加速度（米/秒²），快速达到目标速度
21  acc_lim_theta: 0.5            # 角加速度（弧度/秒²），限制转向速度
22  acc_lim_y: 0.0                # 横向加速度（差速机器人设为0，禁止横向移动）
23
24  # ----- 3. 轨迹生成与避障参数（弱化避障干扰） -----
25  sim_time: 1.0                 # 轨迹模拟时间（秒），不宜过长
26  sim_granularity: 0.05         # 轨迹采样粒度（米），越小轨迹越精细
27  vx_samples: 3                 # 线速度采样数量，减少冗余选项
28  vtheta_samples: 5             # 角速度采样数量，限制转向选择
29
30  occdist_scale: 0.5            # 障碍物距离权重，调低减少避障对直线的影响
31  publish_cost_grid_pc: false   # 关闭成本网格发布，减少计算消耗
32  meter_scoring: true           # 启用米制评分，确保参数单位统一
33
34  # ----- 4. 其他辅助参数 -----
35  prune_plan: true              # 修剪规划路径，去除机器人后方冗余点
36  recovery_behavior_enabled: false # 关闭恢复行为，避免异常时偏离直线

```

表解释该参数文件中的关键变量的含义和用途：

参数名	含义	用途
path_distance_bias:	机器人跟踪全局路径时，	值越高，机器人越优先保证沿路径行

32.0	“贴合路径”的权重系数	驶。此处设为 32.0，是强化直线跟踪的核心参数，确保机器人不轻易偏离预设直线
goal_distance_bias: 0.8	机器人跟踪目标点时，“靠近目标”的权重系数	值较低（0.8），表示降低对“快速到达目标”的优先级，避免为了赶向终点而偏离直线路径
xy_goal_tolerance: 0.15	到达目标点时，允许的XY方向最大误差（米）	0.15 米表示机器人到达目标附近 ± 0.15 米范围内即视为达标，值较小可保证直线终点的精度，但过小将增加调整负担，导致原地微调
yaw_goal_tolerance: 0.16	到达目标点时，允许的朝向（偏航角）最大误差（弧度， $\approx 9.17^\circ$ ）	确保机器人到达终点时的朝向符合直线方向要求，值较小可提升方向精度，避免终点朝向偏移
max_vel_x: 0.7	机器人最大前进线速度（米/秒）	限制最高速度，确保在直线跟踪时的稳定性
min_vel_x: 0.1	机器人最小前进线速度（米/秒）	强制机器人保持最低移动速度，避免因“频繁启停”导致的路径偏移
max_vel_theta: 1.2	最大旋转角速度（弧度/秒， $\approx 68.8^\circ/\text{秒}$ ）	限制转向速度，避免因快速旋转导致的方向超调
min_vel_theta: -1.2	最小旋转角速度（负号表示反向旋转）	与 max_vel_theta 对称，确保机器人可双向旋转调整方向
min_in_place_rotational_vel: 1.0	原地旋转时的最小角速度（弧度/秒）	确保机器人需要原地调整方向时，能快速达到足够的旋转速度，避免卡顿
acc_lim_x: 1.0	线加速度限制（米/秒 ² ）	控制加速快慢，值较大（1.0）可让机器人快速达到目标速度，减少加速阶段的路径偏移
sim_time: 1.0	轨迹规划时模拟的未来时间（秒）	值较小（1.0）可减少计算量，同时让规划更关注近期路径
vx_samples: 3	线速度的采样数量	数量较少可减少冗余计算，避免因速度波动导致的路径偏移
vtheta_samples: 5	角速度的采样数量	数量适中（5），既保证一定的转向调整能力，又不引入过多旋转选项
occdist_scale: 0.5	障碍物距离的权重系数，值越高，机器人越倾向于远离障碍物	值较低（0.5）表示弱化避障优先级，优先保证直线跟踪

这套参数通过强化路径贴合权重、限制转向干扰、弱化非必要功能，实现“严格跟踪直线”的目标。核心逻辑是：让机器人优先服从路径方向，减少转向调整的频率和幅度，同时保持稳定的前进速度，适合无复杂障碍物、对直线精度要求高的场景。

（4）多点导航任务调度（multi_nav.py）：

核心逻辑：通过multi_nav.py脚本实现任务协调，控制机器人一次导航到多个预设目标点（waypoints），在某些点上停下来等待几秒并自动拍照保存到指定文件夹，拍照后继续前往下一个目标点，直到所有点都完成。

代码解析：定义了一个NavigationPhotoBot类，核心作用是封装机器人导航与拍照的完整功能，将相关的数据（如航点、图像、计数器等）和操作（如导航、拍照、图像处理等）组织在一起，形成一个模块化的功能单元。以下为完整代码流程：

①首先定义了初始化方法(_init_)，其结构如下所示：

初始化 (_init_)

- └─ ROS节点初始化：创建名为nav_photo_bot的节点
- └─ 航点存储：接收外部传入的航点列表（waypoints）
- └─ 图像保存配置：
 - └─ 使用CvBridge实现 ROS 图像消息与 OpenCV 格式的转换
 - └─ 通过线程锁（threading.Lock）保证图像数据读写安全
 - └─ 订阅摄像头话题，等待第一帧图像接收成功
- └─ 导航客户端初始化：
 - └─ 创建move_base的 Action 客户端，用于发送导航目标
 - └─ 等待move_base服务连接成功
- └─ 拍照计数配置：
 - └─ 映射3个拍照点到文件夹1, 2, 3
 - └─ 记录每个文件夹的照片数量

如下开始定义核心方法：

②首先定义图像回调函数(image_callback)，其结构和代码如下所示：

图像回调(image_callback)：

- └─ 订阅摄像头话题的回调函数，实时更新最新图像
- └─ 通过线程锁确保多线程环境下图像数据的安全性
- └─ 将 ROS 的Image消息转换为 OpenCV 的bgr8格式（适合保存为图片）

```
#摄像头图像回调函数
def image_callback(self, msg):
    with self.image_lock:
        try:
            # 将ROS Image消息转换为OpenCV (bgr8) 格式
            self.latest_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        except CvBridgeError as e:
            rospy.logerr(f"CvBridge 错误: {e}")
```

③定义**角度转换函数degrees_to_quaternion**，将欧拉角（偏航角，单位度）转换为四元数，用于设置机器人导航目标的朝向（ROS 中姿态表示需用四元数），如下为源码：

```
#将YAW偏航角(角度)转换为四元数
def degrees_to_quaternion(self, yaw_deg):
    yaw_rad = math.radians(yaw_deg)
    q = quaternion_from_euler(0, 0, yaw_rad)
    return Quaternion(q[0], q[1], q[2], q[3])
```

④定义**图像保存函数(save_current_image)**，其结构如下所示：

图像保存函数(save_current_image)

- └─ 根据文件夹名称（1/2/3）保存当前图像

- └─ 自动创建目录（根目录robot_captures及子文件夹）
- └─ 生成有序文件名（img_0.png、img_1.png等）
- └─ 使用 OpenCV 的imwrite保存图像，并更新计数器

⑤接着是**任务执行函数(run_mission)**，其结构如下所示，代码见附录：

任务执行函数(run_mission)

- └─ 整体框架：遍历航点列表
- └─ 单航点处理流程：
 - └─ 日志输出与航点信息展示
 - └─ 构建导航目标（MoveBaseGoal），包含坐标系、时间戳、位置和姿态
 - └─ 通过move_base客户端发送目标，并等待机器人到达
 - └─ 根据move_base返回的状态，判断是否成功到达，并执行后续动作
- └─ 任务结束处理：当所有航点遍历完成后，输出任务结束日志

⑥最后编写**主程序**（if __name__ == '__main__'），其结构和代码如下所示：

主程序（if __name__ == '__main__'）

- └─ 定义航点列表（mission_waypoints）：
 - └─ 每个航点包含位置（x、y）、朝向（yaw_deg）
 - └─ 标记需要拍照的航点（action: 'photo'）
- └─ 创建NavigationPhotoBot实例并执行任务（run_mission）
- └─ 捕获 ROS 中断异常和其他错误，确保程序稳定退出

```
if __name__ == '__main__':
    try:
        #航点列表
        mission_waypoints = [
            # 出生点
            {'x': 1.6856, 'y': -1.6708, 'yaw_deg': 90},
            {'x': 1.6356, 'y': 1.7380, 'yaw_deg': 180},
            {'x': 0.4414, 'y': 1.7080, 'yaw_deg': -90},
            # 拍照点 1: 保存到 A 文件夹
            {'x': 0.386, 'y': 1.1, 'yaw_deg': -14, 'action': 'photo'},
            # 拍照点 2: 保存到 B 文件夹
            {'x': 0.6741, 'y': 0.9917, 'yaw_deg': 185, 'action': 'photo'},
            # 过渡点...
            {'x': 0.6741, 'y': 0.9917, 'yaw_deg': -90},
            {'x': 0.5705, 'y': -0.030, 'yaw_deg': 180},
            {'x': -1.7334, 'y': 0.0189, 'yaw_deg': -90},
            {'x': -1.7334, 'y': -1.155, 'yaw_deg': 0},
            # 拍照点 3: 保存到 C 文件夹
            {'x': -1.0894, 'y': -1.051, 'yaw_deg': -85, 'action': 'photo'},
            # 过渡点
            {'x': -1.0894, 'y': -1.051, 'yaw_deg': 0},
            {'x': 1.7075, 'y': -1.1755, 'yaw_deg': -90},
            {'x': 1.6856, 'y': -1.6708, 'yaw_deg': 90},
        ]

        bot = NavigationPhotoBot(mission_waypoints)
        bot.run_mission()

    except rospy.ROSInterruptException:
        rospy.loginfo("导航拍照任务被中断。")
    except Exception as e:
        rospy.logerr(f"发生未预料到的错误: {e}")
```

3.4.3 难点与对策：

难点：如何配置正确的参数组合（局部规划器参数+代价地图参数）来保证机器人行驶的效果

对策：以下为参数调节的总体思路，可以根据这个思路逐步调节参数，要实现不同的目标需要调整对应的参数组合，最终达到在避障响应及时的情况下保证机器人有最好的行驶效果：

调整目标	优先调整参数	建议方向
避障更安全、更平滑	inflation_radius ↑ cost_scaling_factor ↓ sim_time ↑	增大安全区、路径更平滑但更保守
动作更灵敏、行进更快	sim_time ↓ vx_samples ↓ cost_scaling_factor ↑	响应快但路径可能折线、贴边
定位更精准（到点精度高）	xy_goal_tolerance ↓ yaw_goal_tolerance ↓	提高到点精度但可能在目标点反复微调
计算压力过大（CPU占用高）	sim_time ↓ vx_samples ↓ vtheta_samples ↓ obstacle_range ↓	降低采样与感知范围
路径跟踪更贴合直线、精度更高	path_distance_bias ↑ goal_distance_bias ↓	机器人严格沿全局直线行驶，到达目标时位姿更精准

3.5 关键模块三：数据应用与输出层

3.5.1 功能描述

该模块主要对模块二采集到的数据（图片）进行应用。人员和车牌识别功能是智慧社区巡检系统最终价值输出的核心算法支撑。其中，人员识别功能通过yolo_people.py脚本实现，该脚本专门针对人员检测点的采集图像进行离线批量处理，利用预训练的YOLOv5模型对目标行人进行精准检测与定位，并根据其类别标签完成社区内/外部人员的分类计数，最终将带有标注的图像和统计结果汇总输出。

车辆识别功能则由detect_plate.py脚本承担，它集成了基于YOLOv5架构训练的专门的车牌检测模型以及专门设计的字符识别网络--plate_recognition/plateNet.py中定义的CNN+RNN结构，对车辆识别区的采集图像或视频流进行分析，实现车牌的精确框定和号码提取。这两个脚本均采用离线执行的方式，读取机器人自主导航过程中采集到本地的图像数据进行处理，确保高负荷的深度学习推理运算不影响机器人导航系统的实时稳定运行。

3.5.2 算法原理与实现方法

（1）社区人员识别(yolo_people.py)

Yolo_people.py脚本是一个基于YOLOv5的社区人员识别批量处理工具，核心功能是对指定文件夹中的图像进行批量检测，统计“社区内部人员（nei）”和“社区外部人员（wai）”的数量，并保存带检测框的图像结果。代码支持两个场景（街道A、街道B）的独立识别与

统计，通过加载本地训练好的YOLOv5模型（`best.pt`），自动适配GPU/CPU设备，批量处理图像后输出分类计数结果，同时将可视化结果保存到指定文件夹，适用于智慧社区的人员流动监测场景。

yolo_people.py脚本主要由两部分组成，分别是核心函数batch_detect_yolov5定义和主程序（if __name__ == '__main__':）这两部分。

核心函数(batch_detect_yolov5)结构如下所示：

核心函数(batch_detect_yolov5)：

- └─ 防御性检查：输入合法性校验
 - └─ 检查模型权重文件（model_path）是否存在
 - └─ 检查待检测图像文件夹（image_dir）是否存在
 - └─ 检查本地 YOLOv5 代码库路径（YOLOV5_REPO_PATH）是否正确
- └─ 核心依赖准备：YOLOv5 模型加载并自动选择GPU/CPU
- └─ 数据输入准备：待检测图像收集
- └─ 核心检测逻辑：批量推理执行
 - └─ 调用model(image_paths)对批量图像进行推理
 - └─ 输出results对象，内含所有图像的检测信息（每张图像对应一个子结果）
 - └─ 记录推理耗时，便于评估效率
- └─ 处理识别结果：结果统计
 - └─ results.pandas().xyxy将结果转为Pandas数据框列表
 - └─ 根据mode区分场景，遍历数据框，累计name为nei和wai的数量
- └─ 结果输出：保存到指定路径
 - └─ 创建输出目录（output_dir，若不存在则自动创建）
 - └─ 调用results.render()生成带检测框的图像数组（RGB 格式）
 - └─ 转换为 OpenCV 支持的 BGR 格式，按原图文件名保存到output_dir
 - └─ 打印保存路径，提示用户查看结果

通过上述结构，核心函数batch_detect_yolov5拥有诸多优点，其实用性强，专为社区人员识别场景设计，支持多区域独立统计并能保存可视化结果；灵活性高，核心函数封装逻辑便于扩展新场景，且支持GPU加速提升效率；稳定性好，通过路径校验和异常处理避免崩溃，适配离线环境。

主程序（if __name__ == '__main__':）的结构如下所示：

主程序（if __name__ == '__main__':）：

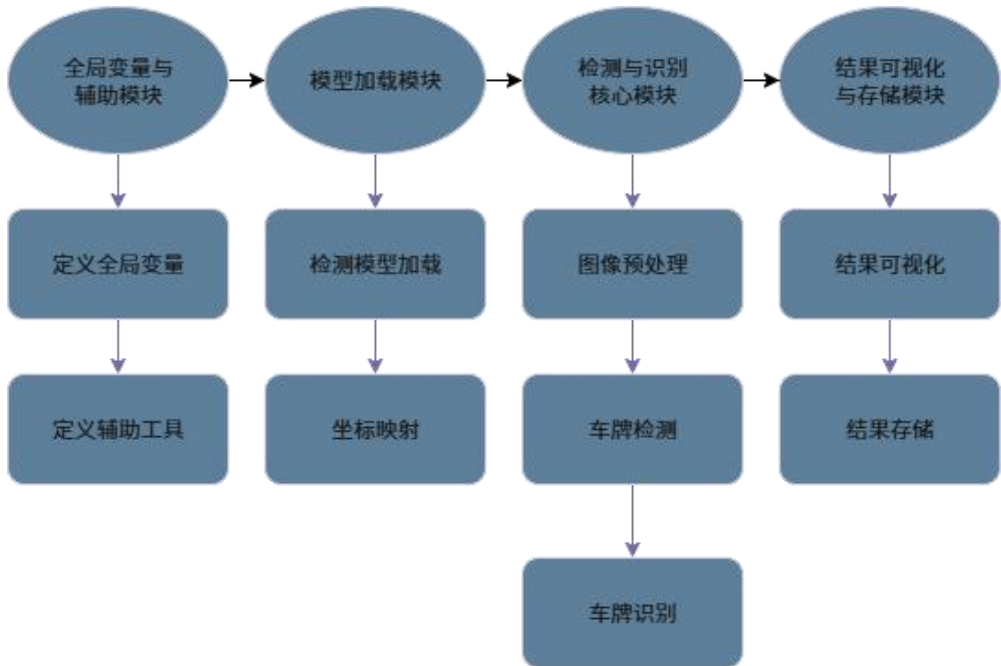
- └─ 路径参数配置：模型路径，输入图像路径和输出结果路径
- └─ 分场景调用核心函数
 - └─ 调用batch_detect_yolov5函数处理街道 A 的图像，传入对应路径和模式
 - └─ 同样调用该函数处理街道 B 的图像，传入对应路径和模式
 - └─ 将两个场景的返回结果（人员统计信息）存入列表all_results
- └─ 汇总并输出总结果

主函数简洁聚焦，负责配置路径、调用核心函数处理不同场景，最后汇总输出结果，易扩展且逻辑清晰。

（2）车牌检测与识别(detect_plate.py)

该脚本是一个基于YOLOv5的车牌检测与识别一体化工具，核心功能是对指定路径的图像或视频进行批量处理，先定位车牌位置及角点坐标，再识别车牌号、车牌颜色（可选）及单/双层类型，同时保存带检测框、关键点和识别结果的可视化图像（或视频）。代码支持单张/批量图像、视频三种处理场景，通过加载本地训练的车牌检测模型（plate_detect.pt）和识别模型（plate_rec_color.pth），自动适配GPU/CPU设备，批量处理后不仅输出可视化结果，还将识别信息（图像名、车牌号、置信度、坐标等）整理为CSV文件，适用于智慧社区、停车场等场景的车牌信息采集与管理需求。

核心函数部分结构如下图所示：



①全局变量与辅助模块：

该模块是脚本的基础支撑部分，包含“定义全局变量”和“定义辅助工具”两个子环节。
定义全局变量：设置模型路径、图像/视频输入输出路径、检测参数（如置信度阈值、图像尺寸）等全局参数，为后续模块提供统一配置；
定义辅助工具：实现如四点透视变换(four_point_transform)、坐标映射(scale_coords_landmarks)等工具函数，解决车牌区域提取、坐标还原等关键辅助任务，为检测与识别环节提供技术支持。

②模型加载模块：

该模块负责“检测模型加载”和“坐标映射”两个核心动作，是实现车牌检测识别的前提。
检测模型加载：通过attempt_load函数加载本地训练的车牌检测模型（如plate_detect.pt），并自动适配CPU设备，为后续推理提供模型支持；
坐标映射：通过工具函数将模型输出的检测框、车牌角点坐标从模型输入尺寸还原到原始图像尺寸，确保定位结果与原图的空间一致性。

③检测与识别核心模块：

该模块是脚本的功能核心，包含“图像预处理”“车牌检测”“车牌识别”三个递进环节。
图像预处理：对输入图像进行尺寸调整、格式转换（BGR→RGB、维度重排）和归一化，使其符合模型输入要求；

车牌检测：利用加载的检测模型对预处理后的图像进行推理，输出车牌的检测框、置信度、角点坐标及类别（单/双层车牌）；

车牌识别：对检测到的车牌区域进行透视变换（双层车牌需分割拼接），再通过识别模型（如plate_rec_color.pth）提取车牌号、颜色（可选）及字符置信度，完成从“定位”到“识别”的核心任务。

④结果可视化与存储模块

该模块负责“结果可视化”和“结果存储”，是功能输出的最终环节。

结果可视化：将车牌检测框、角点、车牌号、颜色等信息绘制在原始图像上，生成带直观标注的可视化结果；

结果存储：一方面将可视化图像（或视频）保存到指定路径，另一方面将识别信息（图像名、车牌号、类型、置信度、坐标等）整理为 CSV 文件持久化存储，满足结果回溯与数据分析需求

模块化设计清晰，功能覆盖车牌检测、识别、预处理及结果输出全流程，且自动适配 GPU/CPU、图像/视频多场景，兼具复用性灵活性，是一套完备且易扩展的车牌识别工具集。

⑤主程序部分：

主程序（if __name__ == '__main__':）：

- └─ 参数配置：通过argparse进行参数配置
- └─ 模型加载：加载车牌检测模型（detect_model）和识别模型（plate_rec_model）
- └─ 分场景处理
 - └─ 图像处理
 - └─ 视频处理
- └─ 汇总并输出总结果

整体逻辑清晰区分图像/视频两种输入类型，兼顾批量处理与实时性，最终通过可视化结果和结构化CSV实现车牌信息的完整输出。

3.5.3 难点与对策

困难：

在项目开始时，我们使用rospy.wait_for_message方法进行同步拍照时，出现了所有保存的照片都是相同（第一次拍照结果）的错误现象。这种机制的问题在于，它会获取ROS话题消息队列中的第一条可用消息，而没有清除或忽略机器人在导航过程中产生的旧图像数据。因此，在机器人到达第二个甚至第三个拍照点并调用wait_for_message时，它很可能只是立即接收了之前在第一个拍照点附近相机采集并仍滞留在队列中的缓存帧，而不是机器人停稳后相机捕捉到的新画面，导致照片内容重复。

对策：

我们采用了更健壮的异步订阅机制来解决这一问题。它通过在初始化时启动一个rospy.Subscriber来持续、实时地接收摄像头图像数据，并将其存储在一个受threading.Lock保护的类变量self.latest_image中。这样，当机器人成功到达任何一个拍照点后，脚本只需要短暂等待机器人稳定（例如1秒），然后直接从这个持续更新的类变量中读取并保存最新的图像帧即可。这种方法确保了每次拍照动作都能获取到机器人停稳后相机实时捕捉的最新画面，从而避免了消息缓存导致的图像重复问题，实现了正确的导航拍照流程。

3.6 本项目创新点

3.6.1 导航参数精细化配置与定点停靠精度优化创新

为满足“机器人在指定任务点精准停靠以采集图像”的核心需求，突破传统导航参数“经验化调试”的局限，构建目标导向型参数优化体系：

(1) **路径跟踪优化**：通过将局部规划器 `path_distance_bias` 参数提升至 32.0、降低 `goal_distance_bias` 至 0.8，强化机器人对全局直线路径的贴合优先级，避免为“赶目标点”而偏离预设路线；

(2) **停靠精度控制**：将 `xy_goal_tolerance`（XY 轴误差）设为 0.15m、`yaw_goal_tolerance`（朝向误差）设为 0.16 弧度，结合 KLD-Sampling 自适应粒子数量的 AMCL 定位（粒子数 500-5000 动态调整），将定位误差控制在 0.05m 以内，确保机器人在人员区域、车辆识别区的停靠位置与角度精准匹配图像采集需求；

(3) **避障与流畅性平衡**：通过局部代价地图 `inflation_radius=0.14m`（小膨胀半径）+`cost_scaling_factor=5.0`（陡峭代价衰减）的组合，既避免障碍物附近区域过度占用，又减少机器人频繁转向，实现避障安全与直线行驶流畅性的最优平衡。

3.6.2 图像采集机制与视觉识别架构解耦创新

针对传统“导航中实时识别”导致系统资源占用过高、导航卡顿的问题，创新设计“采集-识别解耦”架构：

(1) **异步图像采集优化**：摒弃易导致图像重复的 `rospy.wait_for_message` 同步机制，采用线程锁保护的实时图像订阅方法，通过 `rospy.Subscriber` 持续接收摄像头话题数据，将最新帧存储于 `self.latest_image` 变量，机器人停靠后直接读取该变量保存图像，确保每次采集均为停稳后的最新画面，解决多任务点拍照内容重复的核心问题；

(2) **离线批量识别**：机器人巡航时仅完成图像采集与存储，待所有任务点遍历结束后，再启动 `yolo_people.py`（人员识别）、`detect_plate.py`（车牌识别）进行离线批量处理。该架构一方面降低导航过程中的计算负荷，保障机器人移动稳定性；另一方面可集中利用 GPU 资源进行模型推理，提升多图像、多目标的识别效率，同时便于统一汇总识别结果（如生成 CSV 统计文件）。

4. 测试与成果分析

4.1 功能测试

(1) 基础环境与自主导航功能

整个系统功能的验证，首先从底层环境和导航链的稳定运行开始。我们通过启动集成后的 `test.launch` 文件，成功在 Gazebo 中加载了高保真度的社区仿真场景和集成了激光雷达、摄像头传感器的机器人模型，确保了传感器数据话题流的稳定输出。定位环节，系统基于前期 GMapping 建立的静态地图，由 AMCL 算法驱动粒子云迅速收敛，将机器人的定位误差控制在 0.05m 以内的极小范围，为后续的自主移动奠定了坚实基础。随后，通过向 `move_base` 导航栈

发送目标点，机器人均能有效规划出可执行的全局路径，并由局部规划器引导车体平稳、准确地抵达目的地。

（2）巡航任务执行与环境适应能力

项目的核心在于机器人的全自动多点巡检能力。我们运行daohang.py主控制脚本，验证了机器人能够严格按照预设的航点顺序执行任务，包括所有人员识别区和车牌识别区。在巡航过程中，系统充分展示了对复杂环境的适应性：当局部代价地图实时检测到动态障碍物时，导航模块能及时、平滑地调整避障轨迹，在不发生碰撞的情况下绕行，并在脱离障碍物后迅速回归原定的全局路径。更关键的是，在抵达每个预定的巡检点时，机器人均能实现高精度的定点停靠，将XY轴的停靠误差控制在0.15m范围内，这是准确触发后续图像采集的关键前置条件。

（3）智能识别成果

巡检任务完成后，系统自动进入离线智能识别环节，形成完整的功能闭环。在巡线时，机器人采集并保存了高质量图像。接着，我们运行离线检测脚本yolo_people.py，验证了识别模块的有效性。该脚本成功加载并利用预训练的YOLOv5模型，精确地完成了图像中的人员检测和社区内外人员的分类计数；然后运行detect_plate.py，它运用用车牌检测模型+识别专用模型，成功且准确地检测并识别出车牌号码，所有检测结果均被清晰汇总并输出，达到了竞赛所要求的全部技术指标。

4.2 性能测试

4.2.1 机器人参数配置测试

为验证不同参数组合对机器人导航性能的影响，我们围绕“路径平滑性”与“避障响应效率”两大核心指标，设计了多组对照实验，对move_base导航栈的关键参数（局部规划器参数、代价地图参数）进行系统性调试与测试，最终筛选出最优参数配置最终具体参数配置见3.4.2章节。现从两个方面对机器人在我们配置的这一套参数下的导航性能进行测试：

（1）路径平滑性测试：

测试项	测试场景	测试结果	结论
两个相隔较远目标点之间机器人直线度测试	观察倒数第三个和倒数第二个目标点之间的行驶情况（无障碍物），重复三次，观察机器人压线次数以及大幅度摇摆次数 三次测试结果均无压线和大幅度震荡情况	仅出现过小幅度震荡但是很快纠正到正确轨迹	该参数下机器人行走的直线度很好，满足任务需求
目标点之间切换的连贯性	让机器人完整跑三次路线，观察机器人临近目标点之前的位置调整和到达目标点时角度调整情况	在机器人三次完整任务路线行驶的过程中仅出现过一次临近目标点时微调时间较长，未出现过到达目标点后原地长时间旋转调整角度的情况	目标点切换情况较好，切换较连贯，满足任务需求

（2）避障性能测试：

测试项	测试场景	测试结果	结论
-----	------	------	----

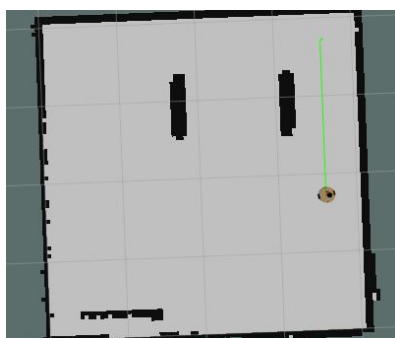
道路一侧有障碍物时机器人的避障程度	让机器人完整跑三次任务路线，观察机器人在遇到道路旁边有障碍物时是否过度避障或者过度贴近障碍物导致偏移路线然后压线	三次完整路线中，仅出现过一次在经过两侧存在障碍物的道路时发生了轻微向无障碍物区域方向震荡但可以及时调整回来	避障程度适中，能够满足机器人行走过程中的避障需求
近障碍物时避障响应速度	观察机器人在靠近障碍物时是否能及时避障，是否未出现避障不及时导致撞到障碍物或者十分靠近障碍物才开始避障的情况，重复三次	机器人靠近障碍物时均及时避障，未出现避障不及时的情况	响应速度及时，满足避障需求

结论：

综合路径平滑性测试与避障测试结果，我们所配置的Trajectory规划器参数表现优异。

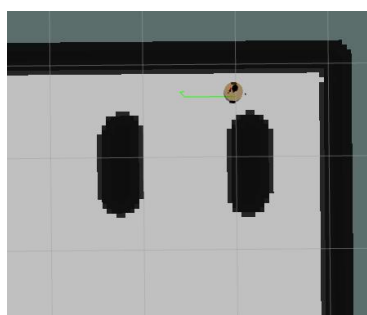
在路径平滑性方面，机器人在无障碍物场景下直线行驶时无压线及大幅度摇摆，仅存在可快速纠正的小幅度震荡，直线度良好；目标点切换过程中仅出现极个别微调时间较长或角度调整频繁的情况，整体连贯性满足任务需求。

如下为机器人路径规划器所规划实时路径：



在避障性能方面，面对道路一侧障碍物时，机器人既能有效避免过度避障或过度贴近障碍物导致的路线偏移，又能在靠近障碍物时及时响应避障，未出现碰撞或避障延迟问题。

如下为两侧有障碍物时的机器人行走情况：



综上，该套参数可确保机器人在智慧社区等场景中实现平稳行驶与安全避障的平衡，完全满足实际任务对路径精度和环境适应性的要求。

4.2.2 视觉模型测试

视觉模块主要有“社区人员识别”和“车牌检测并识别”两大核心任务，社区人员识别任务基于预训练的 YOLOv5 模型，核心目标是精准检测图像中行人，并区分“社区内人员”

与“社区外人员”。车牌检测并识别任务采用“两阶段方案”（第一阶段：YOLOv5 车牌检测模型定位车牌；第二阶段：CNN+RNN 字符识别模型提取号码），核心目标是精准定位车牌区域并识别完整字符。为验证视觉识别模块中我们所训练的模型在智慧社区场景下的实用性与可靠性，现对两项任务的各自模型分别进行测试：

社区人员和车牌识别模型测试：

光照条件	角度条件	人物识别成功率	车牌识别成功率	性能分析与结论
很暗	正面	80%	78%	在极低照度下，模型的特征提取受到挑战，准确率略有下降，但仍保持可用
较暗	正面	100%	99%	在适度低照度下，模型表现稳定，未受明显影响
正常	正面	100%	100%	标准环境光照下，模型达到最佳且稳定性能
较亮	正面	100%	100%	在适度高亮环境中，识别结果保持稳定
很亮	正面	90%	87%	极高亮环境下，可能因局部过曝或反光导致特征丢失，准确率轻微波动
正常	左侧 60°	100%	100%	模型对 60° 倾角具有完全鲁棒性，得益于数据增强或模型的泛化能力
正常	右侧 60°	100%	100%	模型在左右侧 60° 视角下均能实现精确识别
正常	左侧 70°	85%	85%	当拍摄角度增大到 70° 时，透视畸变显著增加，导致识别准确率有所下降
正常	右侧 70°	86%	83%	重复测试结果稳定，印证角度增大到 70° 时，识别准确率有所下降

结论：本项目所采用的社区人员识别模型与车牌检测与识别模型均表现出强大的鲁棒性。在正常、较暗及较亮的主流光照条件下，以及在左右60°的大角度倾斜拍摄时，两个模型的识别成功率均能达到或接近100%。模型的性能边界主要出现在极端条件下，例如光线“很暗”、“很亮”或拍摄角度达到70°时，准确率会因特征提取困难或透视畸变而有所下降。总体而言，这套视觉模型在智慧社区巡检的大多数场景下均能实现精准可靠的识别，完全满足任务需求。

4.3 成果展示

本项目通过一段完整的仿真演示视频，系统地验证了从环境搭建到最终智能识别的全部功能模块。以下表格详细解释了演示视频中的关键画面和对应的系统行为。

模块	重要时间点	关键画面
模块一	00:03 - 00:13	运行命令
	00:13 - 00:26	gazebo 仿真展示
	00:50 - 01:45	slam 建图过程
	01:50 - 02:14	保存地图和展示

模块二	02:21 - 02:28	目录内容处于初始化的空状态
	02:29 - 02:51	运行命令和地图展示
	02:52 - 07:16	导航+拍照
	03:54 - 03:56	小车左转弯（举例）
	03:42 - 05:45	小车右转弯（举例）
	03:24 - 03:25	拍照街道 A 模型
	03:43 - 03:45	拍照街道 B 模型
	04:37 - 04:39	拍照车辆模型
	05:48 - 05:50	车牌识别展示
	05:43 - 05:46	人物识别展示
模块三	05:53 - 08:53	代码流程讲解

5. 总结与展望

5.1 工作总结

本项目围绕“智慧社区”省赛赛题要求，成功设计并实现了一套基于ROS与Gazebo的全自主仿真巡检机器人系统。主要成果如下：

（1）仿真环境搭建：基于SDF与Xacro，我们成功构建了三维仿真世界及机器人模型，并解决了官方模型碰撞体积缺失导致激光雷达失效的关键问题，为后续算法验证奠定了可靠基础。

（2）自主导航实现：实现了从GMapping建图到AMCL定位和move_base导航的完整功能闭环。通过对代价地图、局部规划器等核心参数的精细调整，确保了机器人在复杂环境中的定位精度与路径跟踪稳定性，实现了“定点采集”的核心要求。

（3）任务调度与执行：通过编写daohang.py脚本，实现了多航点的自动任务调度、精准停靠与图像采集功能，实现了全流程自动化运行，为后续的视觉识别提供了识别目标。

（4）高效的视觉识别流程：采用了“采集与识别解耦”的架构，在巡航结束后，利用YOLOv5模型对图像进行离线批量处理，准确高效地完成了行人检测与车牌识别任务。

本系统完整实现了机器人自主巡检与智能化感知的核心功能，达到了赛题预设的各项设计目标。

5.2 未来展望

针对当前方案的局限性，结合智慧社区的实际需求，未来可从以下方面进行改进拓展：

（1）从“离线识别”走向“实时智能”：

当前采用“先采集、后处理”的模式，系统无法实时响应突发事件。未来可对视觉模型进行轻量化或使用TensorRT等工具进行加速，探索在机器人端实现实时目标检测，使其具备及时发现异常（如未经授权人员闯入、人员摔倒）并立即告警的能力。

（2）从“固定巡航”走向“动态调度”：

当前巡航路线基于预设航点，灵活性不足。可开发上层任务调度系统，实现动态路径规划。例如，根据社区安防热力图动态调整巡检重点，或在接收到报警信号后自主规划最短路径前往事发地点。

（3）从“单一感知”走向“多维融合”：

导航主要依赖激光雷达（AMCL），视觉仅用于识别。可探索视觉SLAM（VSLAM）或激光雷达与视觉（IMU）的多传感器融合定位方案（如EKF、LIO-SAM等），进一步提升机器人在复杂或动态环境下的定位鲁棒性。

本项目所验证的技术栈具有显著的通用性和可拓展性。一方面，本仿真系统可作为高价值的“数字孪生”平台。它允许在部署物理机器人之前，对其安防巡检、环境监测或物流配送等任务算法进行低成本、高效率的测试与迭代优化，从而加速技术在真实社区、产业园区及大型楼宇中的落地应用。另一方面，该技术框架可从单机智能平滑拓展至多机器人协同系统。通过引入中央调度或分布式协作机制，未来可实现对大范围空间的网格化自主管理，大幅提升运营效率，为构建全场景、高智能的未来智慧城市提供坚实的技术支撑。

附录:

参考文献:

- [1]陈文轩, 杨风暴, 李菠, 等. 改进YOLOv5s的轻量化无人机航拍小目标识别算法[J/OL]. 电子测量技术, 1-12[2025-10-23]. <https://link.cnki.net/urlid/11.2175.TN.20251021.1825.016>.
- [2]赵贺杨, 褚慧慧, 周宏胭, 等. 基于ROS移动机器人SLAM算法的研究[J]. 自动化应用, 2025, 66(12): 33-37+42. DOI: 10.19769/j.zdhy.2025.12.010.
- [3]郑少康. 基于改进JPS算法和DWA算法的移动机器人路径规划研究[D]. 江西师范大学, 2025. DOI: 10.27178/d.cnki.gjxsu.2025.001998.
- [4]龚利文, 周焕银, 李明桂, 等. 基于ROS的遥控水下机器人建模与仿真[J]. 机电工程技术, 2025, 54(07): 124-130.

核心代码片段:

核心代码片段1: Launch文件

```
<?xml version="1.0" encoding="UTF-8"?>
<launch>
  <param name="robot_description" command="$(find xacro)/xacro $(find
smart_car_simulation)/urdf/car.xacro" />

  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="world_name" value="$(find
smart_car_simulation)/worlds/my_map.world" />
  </include>

  <node pkg="gazebo_ros" type="spawn_model" name="spawn_urdf"
    args="-urdf -model mycar -param robot_description
      -x 1.6856
      -y -1.6708
      -Y 1.5708"
  />

  <!-- 设置地图的配置文件 -->
  <arg name="map" default="nav.yaml" />
  <!-- 运行地图服务器, 并且加载设置的地图-->
  <node name="map_server" pkg="map_server" type="map_server" args="$(find
car_slam)/map/$(arg map)"/>

  <!-- 启动 AMCL 节点 -->
  <include file="$(find car_slam)/launch/amcl.launch" />
  <!-- 运行 move_base 节点 -->
  <include file="$(find car_slam)/launch/path.launch" />
  <!-- 运行 rviz -->
```

```

<node pkg="rviz" type="rviz" name="rviz" args="-d $(find car_slam)/config/nav.rviz"
/>

<node name="joint_state_publisher" pkg="joint_state_publisher"
type="joint_state_publisher" />
<node name="robot_state_publisher" pkg="robot_state_publisher"
type="robot_state_publisher" />
</launch>

```

核心代码片段2: run_mission函数

```

def run_mission(self):
    for i, point in enumerate(self.waypoints):
        if rospy.is_shutdown():
            break

        rospy.loginfo(f"--- 正在前往航点 {i+1}/{len(self.waypoints)} ---")
        rospy.loginfo(f"目标: x={point['x']}, y={point['y']},
yaw={point['yaw_deg']} 度")

        # 1. 创建 MoveBaseGoal
        goal = MoveBaseGoal()

        # 确保"map"与 move_base 全局坐标系 (global_frame) 一致
        goal.target_pose.header.frame_id = "map"
        goal.target_pose.header.stamp = rospy.Time.now()

        # 设置坐标
        goal.target_pose.pose.position = Point(point['x'], point['y'], 0.0)

        # 设置姿态 (朝向)
        q = self.degrees_to_quaternion(point['yaw_deg'])
        goal.target_pose.pose.orientation = q

        #发送目标
        self.move_base_client.send_goal(goal)

        rospy.loginfo("目标已发送, 等待机器人到达...")
        # 等待结果 (设置一个超时时间, 例如 60 秒)
        self.move_base_client.wait_for_result(rospy.Duration(60.0))

        #检查结果
        if self.move_base_client.get_state() == actionlib.GoalStatus.SUCCEEDED:
            rospy.loginfo(f"成功到达航点 {i+1}。")

```

```

# 4. 检查是否有 'action'
if point.get('action') == 'photo':
    rospy.loginfo("检测到 'photo' 动作。")
    # 停稳 1 秒后再拍照
    rospy.sleep(1.0)

    folder_key = self.folder_map.get(self.photo_counter)
    if folder_key:
        rospy.loginfo(f"正在拍照并保存到文件夹: {folder_key}")
        self.save_current_image(folder_key)
        self.photo_counter += 1 # 准备映射到下一个文件夹
    else:
        rospy.logwarn(f"请求拍照, 但未找到文件夹映射 (拍照计数: {self.photo_counter})")

else:
    rospy.logwarn(f"未能到达航点 {i+1}。状态: {self.move_base_client.get_state_text()}")

rospy.loginfo("--- 所有航点已处理完毕。任务完成。 ---")

```

核心代码片段3: batch_detect_yolov5函数部分代码

```

if mode in ['peopleA', 'peopleB']:
    # 模式 1 & 2: 统计 'nei' 和 'wai' 人员数量

    street_name = "街道 A" if mode == 'peopleA' else "街道 B"

    nei_count = 0
    wai_count = 0

    try:
        for df in df_list:
            # 统计 'nei' 的数量
            nei_detections = df[df['name'] == 'nei']
            nei_count += len(nei_detections)

            # 统计 'wai' 的数量
            wai_detections = df[df['name'] == 'wai']
            wai_count += len(wai_detections)

    except Exception as e:
        output_lines.append(f"处理检测结果时发生错误 (请确保已安装 pandas): {e}")

# 收集统计结果
output_lines.append(f"--- {street_name} 人员识别统计 ---")

```

```

        output_lines.append(f"社区内部人员:{nei_count}人")
        output_lines.append(f"社区外部人员:{wai_count}人")

    else:
        output_lines.append(f"警告: 不支持的模式 '{mode}'。请使用 'peopleA', 'peopleB'")

    rendered_images = results.render()

    # 遍历已渲染的图像数组并保存
    for i, img_array in enumerate(rendered_images):
        original_path = image_paths[i]
        filename = os.path.basename(original_path)
        save_path = os.path.join(output_dir, filename)

        # YOLOv5 的 render() 输出通常是 RGB 格式
        img_bgr = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)
        cv2.imwrite(save_path, img_bgr)

    # 返回结果字符串
    return "\n".join(output_lines)

```